SD6103 | DATA SYSTEMS PROJECT

Task 3: Required Queries and Verified Results

A PostgreSQL analysis of Singapore HDB resale prices and hawker-centre accessibility

DATA COVERAGE	DATABASE INSTANCE
January 1990 to October 2026	**976,441 matched transactions**
Five official resale files combined	9,852 flats/blocks; 129 hawkers; 43,036 links

Prepared for the five required Appendix A questions, with Q6 retained as an additional all-table coverage audit.

Group: Transcend Self | Academic Year 2026/27, Semester 1

Database model used

The supplied Chen ER diagram is used without substitution. In the logical mapping, the conceptual **nearby** M:N relationship becomes an associative table containing *flat_id*, *hawker_id*, and *distance_m*. The proximity class is derived through the three-row **proximity_level** reference table.

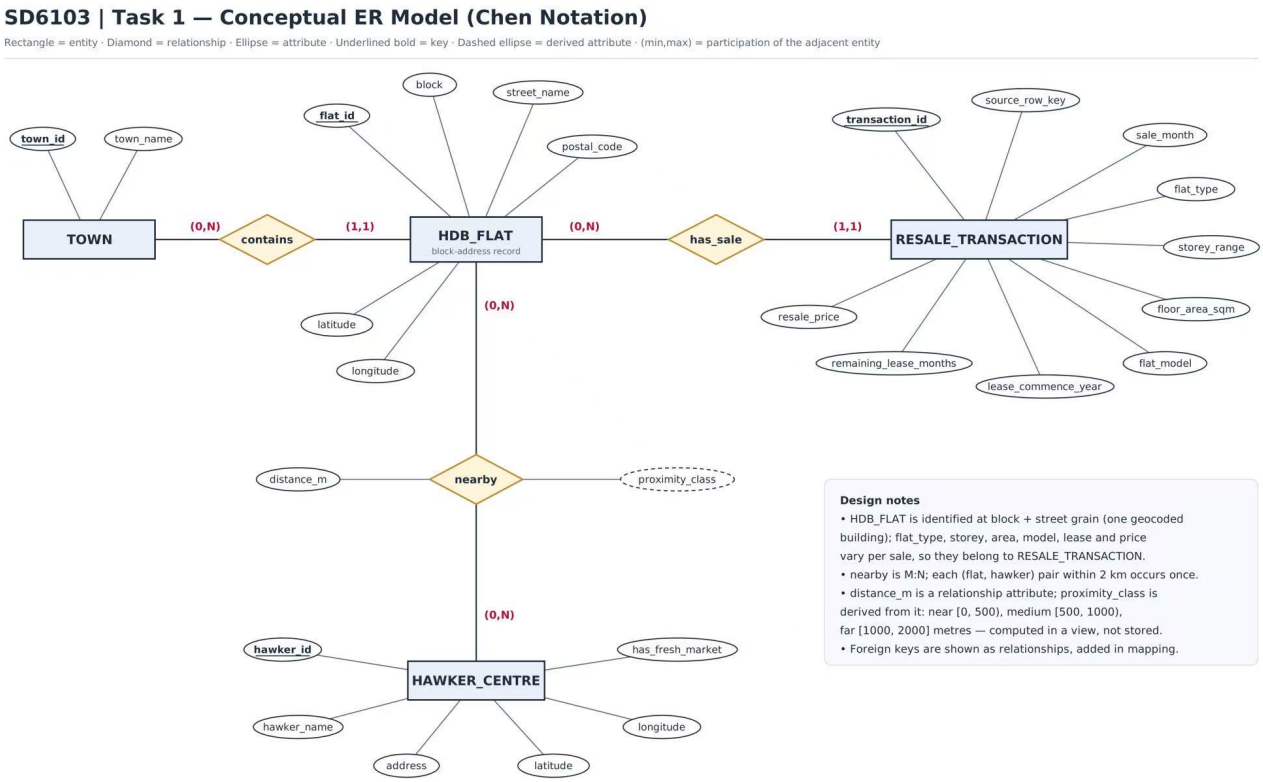


Figure 1. Supplied conceptual ER model (Chen notation).

The source calls the geocoded address record an HDB flat. Because no unit number is available, it is operationally one block-address record; sale-specific attributes remain on RESALE_TRANSACTION.

Data scope and validation

Measure	Value	Definition
Raw resale rows	989,379	All five supplied HDB files
Matched transactions	976,441	Block + street matched to HDB coordinates
Participating flat/block records	9,852	Addresses represented in matched transactions
Hawker centres	129	81 fresh-market yes; 23 no; 25 unknown
Nearby links	43,036	Haversine distance at or below 2,000 m
Latest sale month	2026-10	Determines the Q1 rolling window

Reconciliation with the earlier PDF

The current files contain 3,980 more matched transactions than the earlier report (976,441 versus 972,461), consistent with newly added 2026 records. More importantly, the earlier Task 3 text mixed the complete database with a 1990-1999 subset: its Q2 count of 29 groups, Q3 count of 99 rows, and Q5 SGD 200,000 threshold cannot be reproduced from the complete loaded database. The results below use one consistent scope: all matched transactions, except for Q1's required 36-month filter.

Distance classes: near [0, 500) m; medium [500, 1,000) m; far [1,000, 2,000] m. Distances are straight-line great-circle distances, not walking routes.

Q1 | Price per square metre across towns

The query finds the latest loaded month and applies an exact 36-month window from November 2023 through October 2026. It then calculates *resale_price / floor_area_sqm* for 4-room and 5-room transactions and averages by town and flat type. This yields 52 town-type groups.

Town	Type	Avg. SGD/m²	Transactions
Central Area	5 Room	11,722.31	70
Central Area	4 Room	11,030.61	216
Queenstown	4 Room	10,522.68	825
Queenstown	5 Room	9,919.74	216
Toa Payoh	4 Room	9,643.47	1,098
Bukit Merah	4 Room	9,436.92	1,227
Kallang/Whampoa	4 Room	9,248.03	997
Bukit Timah	5 Room	9,013.74	55
Toa Payoh	5 Room	8,885.60	418
Bukit Merah	5 Room	8,818.62	532

Answer. Central Area 5-room leads at SGD 11,722.31/m², followed by Central Area 4-room at SGD 11,030.61/m² and Queenstown 4-room at SGD 10,522.68/m². These are descriptive nominal-price comparisons; they do not control for lease age, storey, model, or month.

Q2 | Remaining lease versus transaction price

Transactions are classified as **< 50 years**, **50-69 years**, or **70+ years**. For each town-bucket combination, the query returns the median resale price, mean price per square metre, and transaction volume. The complete database produces 64 observed groups.

Town	Lease bucket	Median price	Avg. SGD/m²	Volume
Ang Mo Kio	< 50 years	$430,888.00	5,925.70	64
Ang Mo Kio	50-69 years	$355,000.00	4,709.68	16,291
Ang Mo Kio	70+ years	$179,000.00	2,640.44	37,091
Bedok	< 50 years	$385,000.00	5,755.10	208
Bedok	50-69 years	$360,000.00	4,629.92	19,231
Bedok	70+ years	$206,350.00	2,700.02	49,810
Bishan	< 50 years	$368,000.00	5,353.66	57
Bishan	50-69 years	$660,000.00	6,329.84	3,648
Bishan	70+ years	$378,000.00	3,679.11	18,532
Bukit Merah	< 50 years	$355,000.00	6,037.36	1,054
Bukit Merah	50-69 years	$337,000.00	4,738.44	10,447
Bukit Merah	70+ years	$348,000.00	4,432.05	22,733
Central Area	< 50 years	$410,888.00	6,796.61	99
Central Area	50-69 years	$425,000.00	6,125.75	2,317
Central Area	70+ years	$223,400.00	4,364.07	4,516

Within-town comparisons are the appropriate reading. For example, Ang Mo Kio's median is SGD 430,888 for the small <50-year sample (64 sales), SGD 355,000 for 50-69 years, and SGD 179,000 for 70+ years. The counterintuitive ordering reflects transaction year and building cohort: most 70+ observations come from earlier, lower-price years.

Answer. Remaining lease is associated with price, but it is heavily confounded by sale year, town, flat size, and building cohort. The query is not a causal estimate of the value of one additional lease year.

Q3 | Large 3-room flats and resale prices

First, the ten towns with the largest mean 3-room floor area are selected. Second, the maximum 3-room resale price is returned for every represented town-year. With the complete 1990-2026 history, this yields 369 town-year rows.

Rank	Town	Mean area (m²)	Latest year	Yearly maximum
1	Tampines	72.50	2026	$715,000.00
2	Choa Chu Kang	71.88	2026	$540,000.00
3	Bukit Panjang	71.75	2026	$562,888.00
4	Ang Mo Kio	71.18	2026	$800,000.00
5	Jurong East	70.88	2026	$565,000.00
6	Woodlands	69.75	2026	$600,000.00
7	Clementi	69.74	2026	$890,000.00
8	Yishun	69.49	2026	$625,000.00
9	Bukit Timah	69.05	2026	$548,888.00
10	Bukit Batok	68.87	2026	$605,000.00

Tampines ranks first at 72.50 m², followed by Choa Chu Kang (71.88 m²) and Bukit Panjang (71.75 m²). The accompanying CSV contains every annual maximum, not only the latest-year snapshot shown above.

Answer. Annual maxima deliberately describe the upper tail and are sensitive to rare large, high-storey, or unusual-model flats. They should not be treated as typical transaction prices.

Q4 | Hawker-centre proximity effect

A block may be within 2 km of several hawkers. Counting every link would repeat its transactions, so each block is classified using only its nearest stored hawker. The query then compares the near and far class averages within each town and flat type. Seventy-five groups contain observations in both classes.

Town	Type	Near average	Far average	Difference
Choa Chu Kang	3 Room	$379,557.29	$194,749.02	94.90%
Toa Payoh	2 Room	$161,922.31	$88,500.00	82.96%
Bukit Batok	4 Room	$544,920.72	$307,442.97	77.24%
Geylang	4 Room	$428,261.53	$277,179.26	54.51%
Kallang/Whampoa	4 Room	$471,309.44	$305,644.40	54.20%
Punggol	5 Room	$747,764.40	$497,165.79	50.41%
Bukit Batok	3 Room	$289,311.96	$192,944.23	49.95%
Toa Payoh	4 Room	$478,387.96	$330,292.63	44.84%
Toa Payoh	5 Room	$597,048.07	$415,105.29	43.83%
Geylang	5 Room	$501,582.59	$351,970.67	42.51%

Answer. Choa Chu Kang 3-room has the largest positive difference (94.90%), followed by Toa Payoh 2-room (82.96%) and Bukit Batok 4-room (77.24%). This is an association, not a causal hawker premium: location, age, and housing mix differ between bands.

Q5 | Town-level price near fresh-market hawkers

Only links to hawkers with *has_fresh_market = true* are retained. Transactions are de-duplicated within a proximity band before averaging. On the complete dataset, SGD 200,000 would return all 22 represented towns and therefore fails the assignment rule. A threshold of **SGD 350,000** produces a non-empty, non-universal result: 17 town-band groups across 9 of 22 towns.

Town	Band	Average resale price	Transactions
Bukit Timah	Near	$480,056.25	2,653
Sembawang	Far	$464,185.54	3,985
Bishan	Medium	$456,862.28	17,165
Bishan	Far	$441,955.60	22,237
Central Area	Near	$421,293.81	5,500
Central Area	Medium	$406,073.56	6,215
Bukit Merah	Far	$398,474.99	34,234
Bukit Merah	Near	$397,909.16	29,362
Bukit Merah	Medium	$393,784.24	30,945
Central Area	Far	$393,707.05	6,932
Marine Parade	Near	$366,195.54	7,271
Kallang/Whampoa	Far	$355,683.18	28,919
Bukit Timah	Far	$354,580.95	976
Tampines	Far	$353,249.65	63,175
Marine Parade	Far	$353,237.40	5,614
Marine Parade	Medium	$352,278.93	6,219
Woodlands	Far	$351,888.66	23,880

Answer. The leading groups are Bukit Timah-near (SGD 480,056.25), Sembawang-far (SGD 464,185.54), and Bishan-medium (SGD 456,862.28). High averages occur in all three bands, so fresh-market access is not confined to the closest class.

Q6 | Additional all-table coverage audit

This additional query joins all six logical tables: TOWN, HDB_FLAT, RESALE_TRANSACTION, NEARBY, HAWKER_CENTRE, and PROXIMITY_LEVEL. It reports distinct flats, distinct transactions, and mean resale price for every town-band-market-status combination. De-duplication ensures that averages are not weighted by the number of hawkers in the same group. The output contains 154 groups.

Town	Band	Market	Flats	Transactions	Avg. price
Toa Payoh	Near	Unknown	11	260	$1,010,085.91
Bukit Batok	Near	Unknown	53	2,073	$614,911.73
Punggol	Near	Unknown	23	1,295	$585,626.60
Punggol	Medium	Unknown	220	9,206	$559,268.08
Bukit Timah	Near	Unknown	19	1,677	$553,081.83
Sengkang	Near	Unknown	200	11,030	$520,308.68
Punggol	Far	Unknown	455	23,451	$513,289.46
Pasir Ris	Near	Unknown	39	2,686	$497,436.12
Bedok	Far	Unknown	13	1,211	$497,144.48
Central Area	Far	Unknown	27	3,143	$497,054.46
Bukit Timah	Near	Yes	26	2,653	$480,056.25
Clementi	Far	Unknown	6	813	$477,880.81
Bishan	Far	Unknown	15	1,171	$472,570.57
Sengkang	Medium	Unknown	222	12,843	$471,225.87

Answer. The 154 groups include 57 groups with unknown market status. Retaining these groups exposes incomplete official-attribute matching rather than silently treating unknown as false, making the query useful for coverage and data-quality auditing.

SQL execution evidence and verification status

The current normalized database was loaded into PostgreSQL 18 and executed through pgAdmin 4. The verification query, all six analytical queries, and the performance benchmark completed against the same 976,441-transaction snapshot used in this report.

Evidence item	Status	Detail
Current normalized data	Verified	976,441 matched transactions; 9,852 flats/blocks; 129 hawkers; 43,036 links
Q1-Q6 result counts	Verified	52, 64, 369, 75, 17 and 154 rows/groups respectively
Executable SQL artifact	Verified	SHA-256: 27344a4491c163fd3eb9bacedbfd030d98b9cf5d29be47d0ac6e4c82a8fae9da
Current PostgreSQL run	Verified	PostgreSQL 18 / pgAdmin 4; housing schema and all six tables loaded
Current pgAdmin screenshot	Verified	Import status PASS; all expected table row counts match
Performance observations	Verified	150 unindexed plus 30 indexed observations; five repetitions per query-condition

Current database verification

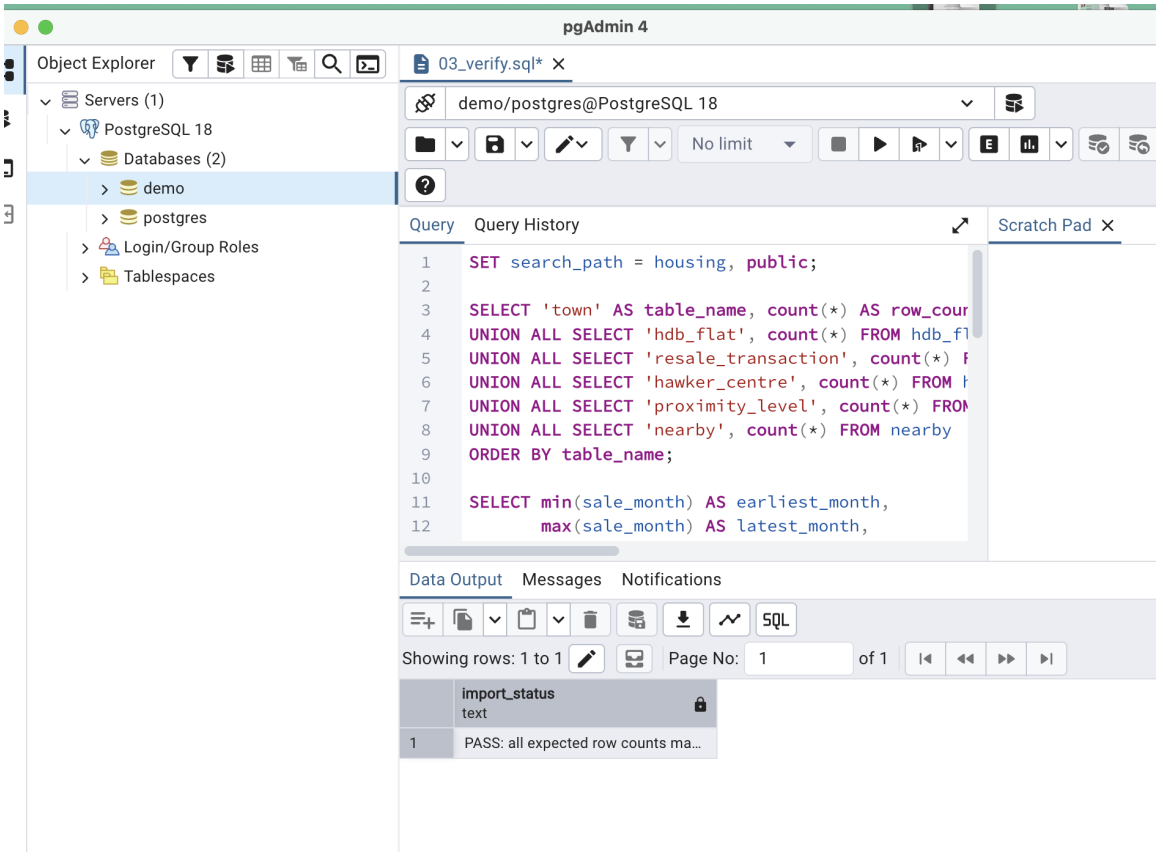
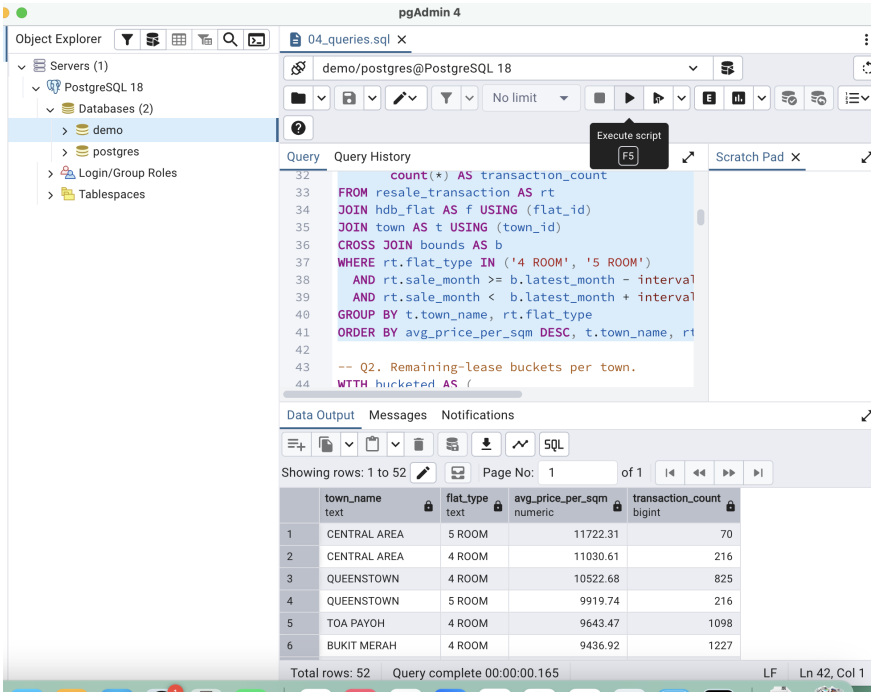


Figure 4. pgAdmin verification query returned PASS: all expected row counts match.

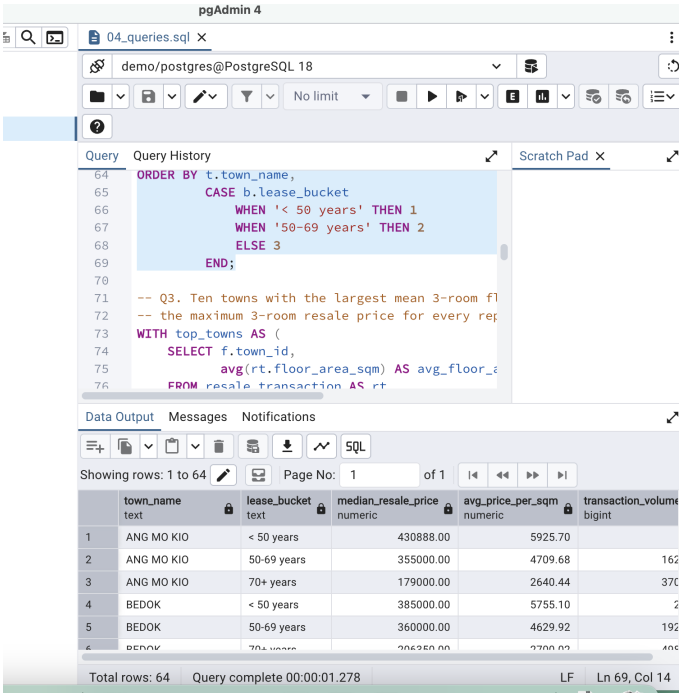
Query	Expected output	Evidence check
Q1	52 groups	Latest month 2026-10; exact 36-month window
Q2	64 groups	Three lease buckets; percentile_cont median
Q3	369 rows	Ten selected towns; annual maximum
Q4	75 groups	Nearest hawker per flat/block; near and far both present
Q5	17 groups / 9 towns	SGD 350,000; fresh-market only; de-duplicated by band
Q6	154 groups	All six tables; yes/no/unknown market status

pgAdmin query evidence | Q1-Q2

Q1: 52 groups

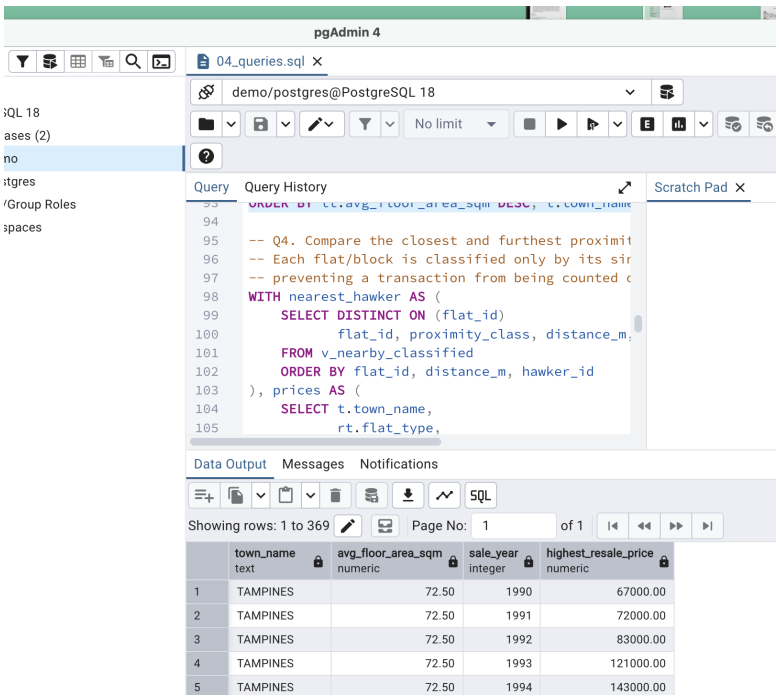


Q2: 64 groups

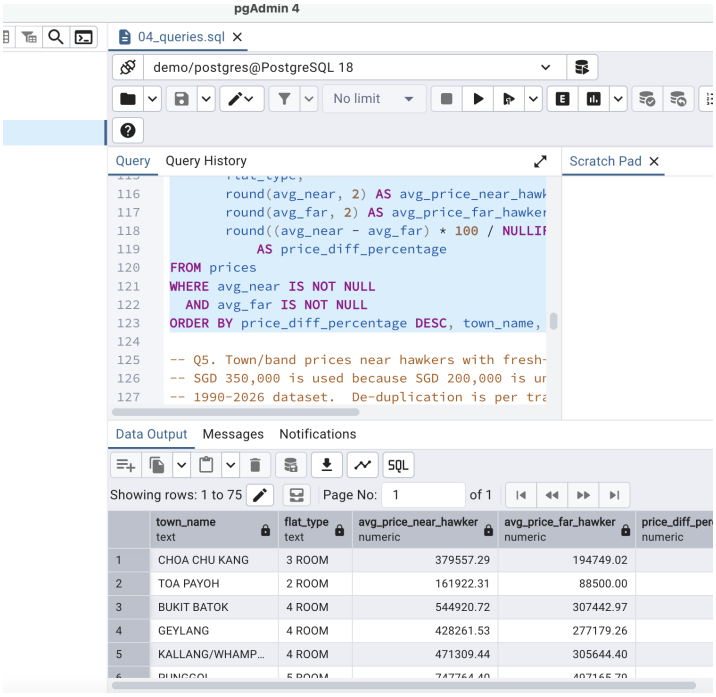


pgAdmin query evidence | Q3-Q4

Q3: 369 rows

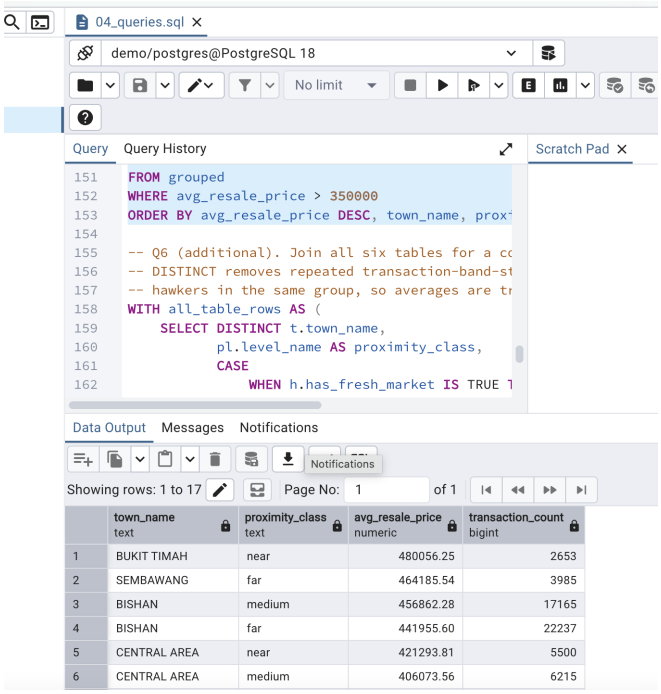


Q4: 75 groups

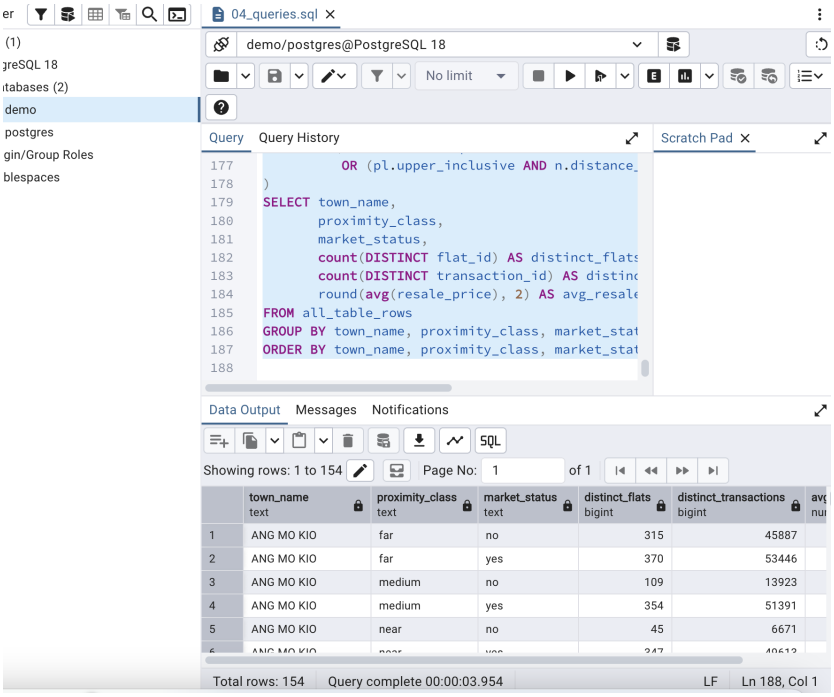


pgAdmin query evidence | Q5-Q6

Q5: 17 groups



Q6: 154 groups; 3.954 s



Performance measurement data

The current PostgreSQL 18 benchmark used five cumulative transaction samples. Each Q1-Q6 statement received one excluded warm-up and five measured repetitions at each size: 5 sizes x 6 queries x 5 repetitions = 150 unindexed observations. At 976,441 rows, the six queries were repeated five times after index creation, adding 30 indexed observations.

Size	Transactions	Benchmark scope
1	278,014	Current cumulative benchmark sample
2	643,914	Current cumulative benchmark sample
3	696,113	Current cumulative benchmark sample
4	733,266	Current cumulative benchmark sample
5	976,441	Current cumulative benchmark sample

Query	Before (ms)	After (ms)	Change	Interpretation
Q1	133.451	18.483	-86.15%	Faster
Q2	105.369	107.803	+2.31%	Slower
Q3	155.387	127.412	-18.00%	Faster
Q4	133.437	134.096	+0.49%	Nearly unchanged
Q5	1,813.712	1,560.546	-13.96%	Faster
Q6	492.608	415.748	-15.60%	Faster

All figures above are current-run PostgreSQL measurements from the same 976,441-row snapshot. Negative percentage change means faster execution after indexes.

Current benchmark: growth across data sizes

Mean execution time is plotted on a logarithmic scale because Q5 is substantially slower than the other queries. The chart uses the 150 measured unindexed observations summarized by query and sample size.

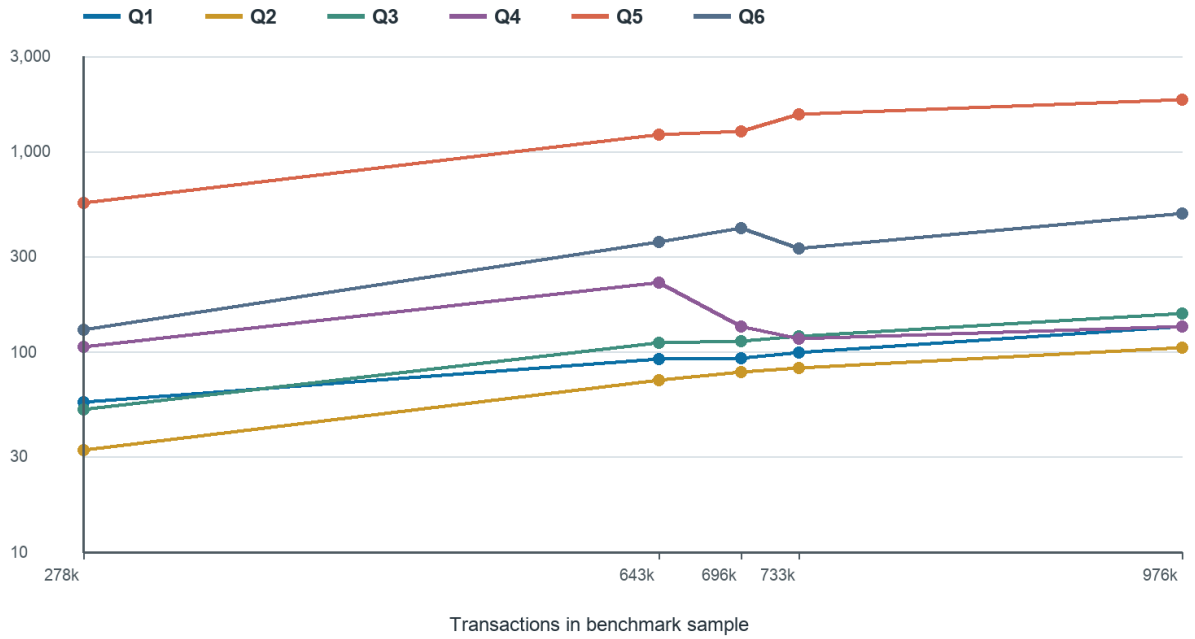


Figure 5. Current PostgreSQL 18 mean execution time across five transaction sizes.

Query	Mean at 976,441 rows	Interpretation
Q1	133.451 ms	Rolling-window filter benefits strongly from the date/type index
Q2	105.369 ms	Median and grouping remain aggregation-bound
Q3	155.387 ms	Town selection and annual grouping scale with history
Q4	133.437 ms	Nearest-hawker classification dominates index gains
Q5	1,813.712 ms	Link join and transaction-band de-duplication are most expensive
Q6	492.608 ms	All-table join and three-dimensional grouping

Current benchmark: index comparison

The grouped bars compare the same largest sample before and after indexes. A logarithmic axis keeps Q1-Q4 visible beside the longer Q5 runtime.

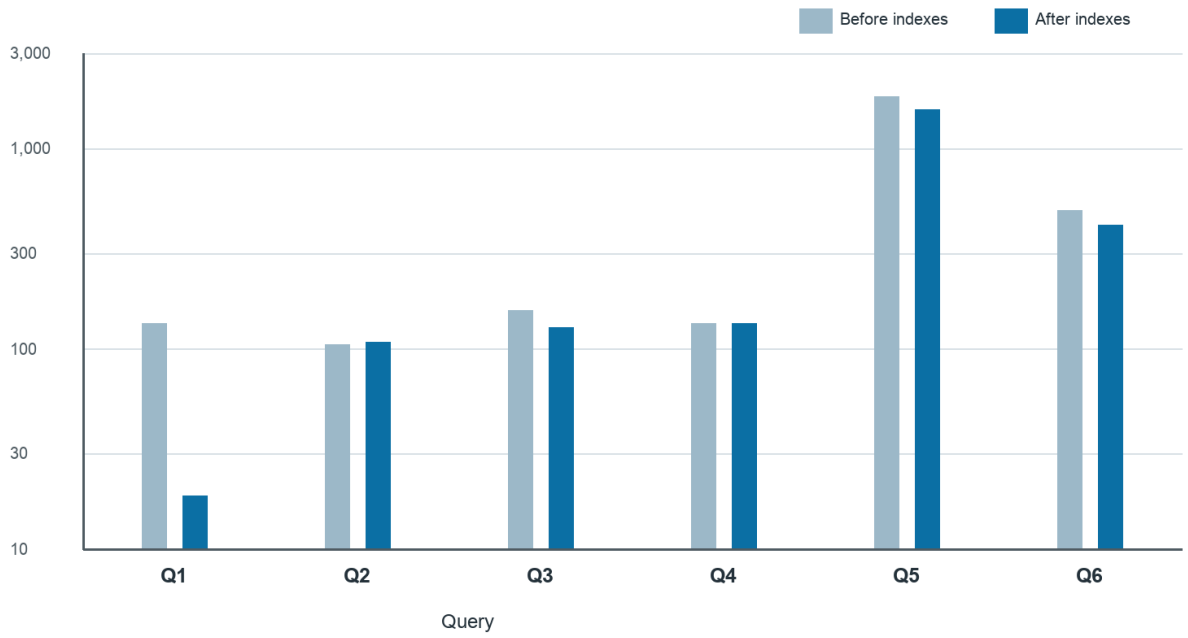


Figure 6. Current before/after index comparison at 976,441 transactions.

Q1 improves by 86.15%, Q3 by 18.00%, Q5 by 13.96%, and Q6 by 15.60%. Q2 becomes 2.31% slower and Q4 changes by only 0.49%, showing that analytical indexes should be justified query by query rather than assumed to help every aggregation.

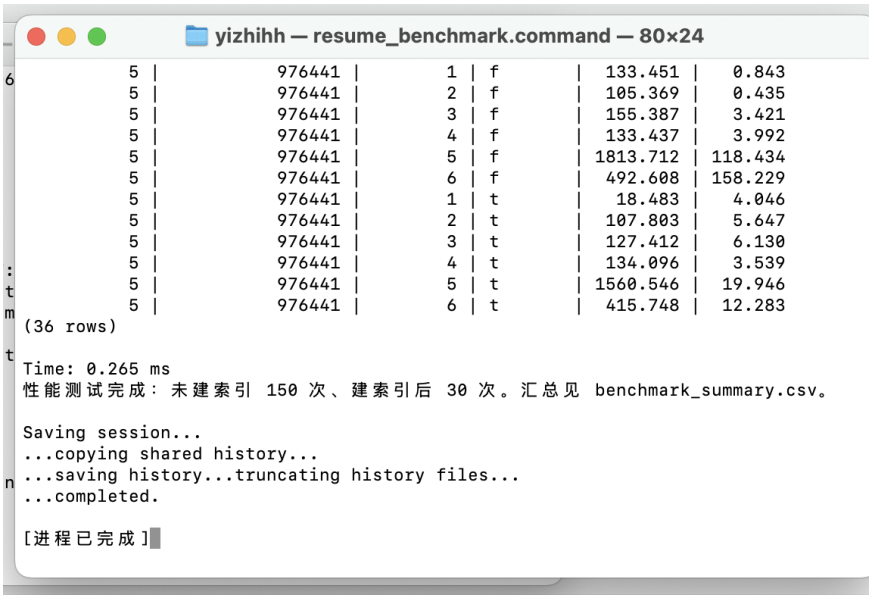


Figure 7. Completed terminal benchmark showing 150 unindexed and 30 indexed observations.

Reproducibility appendix | How to replay the analysis

This appendix places the executable code inside the report, in addition to the separately supplied SQL files. It covers schema creation, CSV loading, integrity verification, Q1-Q6, benchmark measurement, and the indexed benchmark continuation. A reviewer can reproduce the reported outputs from a clean PostgreSQL 18 database by running the scripts in the order below.

Order	Script	Purpose
1	01_schema.sql	Create housing schema and all six normalized tables
2	02_load_data.sql	Load five CSV tables, insert three proximity bands, build indexes and view
3	03_verify.sql	Check row counts, date range, referential coverage, and PASS status
4	04_queries.sql	Execute Q1-Q6 and reproduce all result tables
5	05_benchmark.sql	Collect 150 unindexed observations and create analytical indexes
6	06_resume_indexed_benchmark.sql	Collect 30 indexed observations and export the summary

Exact replay commands

```
cd SD6103_PostgreSQL_Import_Package
./setup_demo.command
./run_queries.command
./run_benchmark.command

# Expected verification status:
# PASS: all expected row counts match
# Expected Q1-Q6 outputs: 52, 64, 369, 75, 17, 154 rows/groups
# Expected benchmark observations: 150 unindexed + 30 indexed
```

The package uses database demo, schema housing, and PostgreSQL user postgres. The scripts stop on the first SQL error. Relative CSV paths are resolved from the package root, so the data and sql directories must remain together.

Appendix A | Schema definition (1/2)

Complete PostgreSQL DDL for the six-table logical model, including keys, foreign keys, checks, and the derived fresh-market flag.

```
001 | \set ON_ERROR_STOP on
002 |
003 | DROP SCHEMA IF EXISTS housing CASCADE;
004 | CREATE SCHEMA housing;
005 | SET search_path = housing, public;
006 |
007 | CREATE TABLE town (
008 |     town_id      integer PRIMARY KEY,
009 |     town_name    text NOT NULL UNIQUE
010 | );
011 |
012 | CREATE TABLE hdb_flat (
013 |     flat_id       bigint PRIMARY KEY,
014 |     town_id       integer NOT NULL REFERENCES town(town_id),
015 |     block         text NOT NULL,
016 |     street_name   text NOT NULL,
017 |     postal_code   char(6),
018 |     latitude      double precision NOT NULL CHECK (latitude BETWEEN -90 AND 90),
019 |     longitude     double precision NOT NULL CHECK (longitude BETWEEN -180 AND 180),
020 |     UNIQUE (block, street_name)
021 | );
022 |
023 | CREATE TABLE resale_transaction (
024 |     transaction_id          bigint PRIMARY KEY,
025 |     flat_id                 bigint NOT NULL REFERENCES hdb_flat(flat_id),
026 |     source_row_key          text NOT NULL UNIQUE,
027 |     sale_month              date NOT NULL,
028 |     flat_type               text NOT NULL,
029 |     storey_range            text NOT NULL,
030 |     floor_area_sqm          numeric(8,2) NOT NULL CHECK (floor_area_sqm &gt; 0),
031 |     flat_model              text NOT NULL,
032 |     lease_commence_year     integer NOT NULL,
033 |     remaining_lease_months  integer NOT NULL CHECK (remaining_lease_months BETWEEN 0 AND
    |     1188),
034 |     resale_price            numeric(14,2) NOT NULL CHECK (resale_price &gt; 0)
035 | );
036 |
037 | CREATE TABLE hawker_centre (
038 |     hawker_id           integer PRIMARY KEY,
039 |     hawker_name         text NOT NULL,
040 |     address             text,
041 |     latitude            double precision NOT NULL CHECK (latitude BETWEEN -90 AND 90),
042 |     longitude           double precision NOT NULL CHECK (longitude BETWEEN -180 AND 180),
043 |     market_stall_count  integer CHECK (market_stall_count &gt;= 0),
044 |     has_fresh_market    boolean GENERATED ALWAYS AS (
045 |         CASE
046 |             WHEN market_stall_count IS NULL THEN NULL
047 |             ELSE market_stall_count &gt; 0
048 |         END
049 |     ) STORED
050 | );
051 |
052 | CREATE TABLE proximity_level (
053 |     proximity_level_id  smallint PRIMARY KEY,
054 |     level_name           text NOT NULL UNIQUE,
055 |     min_distance_m       numeric(8,2) NOT NULL,
056 |     max_distance_m       numeric(8,2) NOT NULL,
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix A | Schema definition (2/2)

```
057 |     upper_inclusive      boolean NOT NULL DEFAULT false,
058 |     CHECK (min_distance_m &gt;= 0 AND max_distance_m &gt; min_distance_m)
059 | );
060 |
061 | CREATE TABLE nearby (
062 |     flat_id      bigint NOT NULL REFERENCES hdb_flat(flat_id),
063 |     hawker_id    integer NOT NULL REFERENCES hawker_centre(hawker_id),
064 |     distance_m   numeric(8,2) NOT NULL CHECK (distance_m &gt;= 0 AND distance_m &lt;= 2000),
065 |     PRIMARY KEY (flat_id, hawker_id)
066 | );
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix B | Data loading and derived proximity view

Complete import script. It loads the packaged CSV files, creates the three distance bands, adds analytical indexes, runs ANALYZE, and creates the reusable classification view.

```
001 | \set ON_ERROR_STOP on
002 | \timing on
003 | SET search_path = housing, public;
004 |
005 | BEGIN;
006 | \copy town (town_id, town_name) FROM 'data/town.csv' WITH (FORMAT csv, HEADER true)
007 | \copy hdb_flat (flat_id, town_id, block, street_name, postal_code, latitude, longitude)
    |     FROM 'data/hdb_flat.csv' WITH (FORMAT csv, HEADER true)
008 | \copy resale_transaction (transaction_id, flat_id, source_row_key, sale_month, flat_type,
    |     storey_range, floor_area_sqm, flat_model, lease_commence_year, remaining_lease_months,
    |     resale_price) FROM 'data/resale_transaction.csv' WITH (FORMAT csv, HEADER true)
009 | \copy hawker_centre (hawker_id, hawker_name, address, latitude, longitude,
    |     market_stall_count) FROM 'data/hawker_centre.csv' WITH (FORMAT csv, HEADER true, NULL
    |     '')
010 | \copy nearby (flat_id, hawker_id, distance_m) FROM 'data/nearby.csv' WITH (FORMAT csv,
    |     HEADER true)
011 |
012 | INSERT INTO proximity_level
013 |     (proximity_level_id, level_name, min_distance_m, max_distance_m, upper_inclusive)
014 | VALUES
015 |     (1, 'near',   0,    500,  false),
016 |     (2, 'medium', 500,  1000, false),
017 |     (3, 'far',    1000, 2000, true);
018 | COMMIT;
019 |
020 | CREATE INDEX idx_hdb_flat_town ON hdb_flat (town_id);
021 | CREATE INDEX idx_resale_month_type ON resale_transaction (sale_month, flat_type);
022 | CREATE INDEX idx_resale_flat ON resale_transaction (flat_id);
023 | CREATE INDEX idx_resale_lease ON resale_transaction (remaining_lease_months);
024 | CREATE INDEX idx_nearby_flat_distance ON nearby (flat_id, distance_m, hawker_id);
025 | CREATE INDEX idx_nearby_hawker ON nearby (hawker_id);
026 | CREATE INDEX idx_hawker_fresh ON hawker_centre (has_fresh_market);
027 |
028 | ANALYZE town;
029 | ANALYZE hdb_flat;
030 | ANALYZE resale_transaction;
031 | ANALYZE hawker_centre;
032 | ANALYZE nearby;
033 |
034 | CREATE OR REPLACE VIEW v_nearby_classified AS
035 | SELECT n.flat_id,
036 |        n.hawker_id,
037 |        n.distance_m,
038 |        pl.proximity_level_id,
039 |        pl.level_name AS proximity_class
040 | FROM nearby AS n
041 | JOIN proximity_level AS pl
042 |   ON n.distance_m &gt;= pl.min_distance_m
043 |  AND (n.distance_m &lt; pl.max_distance_m
044 |       OR (pl.upper_inclusive AND n.distance_m &lt;= pl.max_distance_m));
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix C | Verification checks

Complete verification script used to reconcile table counts and confirm that the imported database matches the report snapshot.

```
001 | SET search_path = housing, public;
002 |
003 | SELECT 'town' AS table_name, count(*) AS row_count FROM town
004 | UNION ALL SELECT 'hdb_flat', count(*) FROM hdb_flat
005 | UNION ALL SELECT 'resale_transaction', count(*) FROM resale_transaction
006 | UNION ALL SELECT 'hawker_centre', count(*) FROM hawker_centre
007 | UNION ALL SELECT 'proximity_level', count(*) FROM proximity_level
008 | UNION ALL SELECT 'nearby', count(*) FROM nearby
009 | ORDER BY table_name;
010 |
011 | SELECT min(sale_month) AS earliest_month,
012 |        max(sale_month) AS latest_month,
013 |        count(DISTINCT date_trunc('month', sale_month)) AS represented_months
014 | FROM resale_transaction;
015 |
016 | SELECT count(*) FILTER (WHERE has_fresh_market IS TRUE) AS fresh_yes,
017 |        count(*) FILTER (WHERE has_fresh_market IS FALSE) AS fresh_no,
018 |        count(*) FILTER (WHERE has_fresh_market IS NULL) AS fresh_unknown
019 | FROM hawker_centre;
020 |
021 | SELECT
022 |     (SELECT count(*) FROM resale_transaction rt LEFT JOIN hdb_flat f USING (flat_id) WHERE
    |     f.flat_id IS NULL) AS orphan_transactions,
023 |     (SELECT count(*) FROM hdb_flat f LEFT JOIN town t USING (town_id) WHERE t.town_id IS
    |     NULL) AS orphan_flats,
024 |     (SELECT count(*) FROM nearby n LEFT JOIN hdb_flat f USING (flat_id) WHERE f.flat_id IS
    |     NULL) AS orphan_nearby_flats,
025 |     (SELECT count(*) FROM nearby n LEFT JOIN hawker_centre h USING (hawker_id) WHERE
    |     h.hawker_id IS NULL) AS orphan_nearby_hawkers;
026 |
027 | SELECT CASE
028 |            WHEN (SELECT count(*) FROM town) = 26
029 |             AND (SELECT count(*) FROM hdb_flat) = 9852
030 |             AND (SELECT count(*) FROM resale_transaction) = 976441
031 |             AND (SELECT count(*) FROM hawker_centre) = 129
032 |             AND (SELECT count(*) FROM proximity_level) = 3
033 |             AND (SELECT count(*) FROM nearby) = 43036
034 |            THEN 'PASS: all expected row counts match'
035 |            ELSE 'CHECK: one or more row counts differ'
036 |        END AS import_status;
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix D | Executable analytical queries Q1-Q6 (1/4)

Complete SQL for every reported analysis. Each query is labelled Q1-Q6 and can be selected and executed independently in pgAdmin Query Tool.

```
001 | -- SD6103 Task 3 - Required analytical queries
002 | -- PostgreSQL 18; schema names follow the supplied Chen ER diagram.
003 | -- Tables: town, hdb_flat, resale_transaction, nearby,
004 | --         hawker_centre, proximity_level.
005 |
006 | SET search_path = housing, public;
007 |
008 | -- Reusable classification view.  The three intervals are:
009 | -- near [0,500), medium [500,1000), far [1000,2000].
010 | CREATE OR REPLACE VIEW v_nearby_classified AS
011 | SELECT n.flat_id,
012 |        n.hawker_id,
013 |        n.distance_m,
014 |        pl.proximity_level_id,
015 |        pl.level_name AS proximity_class
016 | FROM nearby AS n
017 | JOIN proximity_level AS pl
018 |   ON n.distance_m &gt;= pl.min_distance_m
019 |  AND (n.distance_m &lt; pl.max_distance_m
020 |       OR (pl.upper_inclusive AND n.distance_m &lt;= pl.max_distance_m));
021 |
022 | -- Q1. Average price per square metre for 4-room and 5-room flats
023 | -- in the 36-month window ending at the latest loaded month.
024 | WITH bounds AS (
025 |     SELECT date_trunc('month', max(sale_month))::date AS latest_month
026 |     FROM resale_transaction
027 | )
028 | SELECT t.town_name,
029 |        rt.flat_type,
030 |        round(avg(rt.resale_price / NULLIF(rt.floor_area_sqm, 0)), 2)
031 |            AS avg_price_per_sqm,
032 |        count(*) AS transaction_count
033 | FROM resale_transaction AS rt
034 | JOIN hdb_flat AS f USING (flat_id)
035 | JOIN town AS t USING (town_id)
036 | CROSS JOIN bounds AS b
037 | WHERE rt.flat_type IN ('4 ROOM', '5 ROOM')
038 |   AND rt.sale_month &gt;= b.latest_month - interval '35 months'
039 |   AND rt.sale_month &lt;  b.latest_month + interval '1 month'
040 | GROUP BY t.town_name, rt.flat_type
041 | ORDER BY avg_price_per_sqm DESC, t.town_name, rt.flat_type;
042 |
043 | -- Q2. Remaining-lease buckets per town.
044 | WITH bucketed AS (
045 |     SELECT rt.*,
046 |            CASE
047 |                WHEN rt.remaining_lease_months &lt; 600 THEN '&lt; 50 years'
048 |                WHEN rt.remaining_lease_months &lt; 840 THEN '50-69 years'
049 |                ELSE '70+ years'
050 |            END AS lease_bucket
051 |     FROM resale_transaction AS rt
052 | )
053 | SELECT t.town_name,
054 |        b.lease_bucket,
055 |        round(percentile_cont(0.5) WITHIN GROUP (ORDER BY b.resale_price)::numeric, 2)
056 |            AS median_resale_price,
057 |        round(avg(b.resale_price / NULLIF(b.floor_area_sqm, 0)), 2)
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix D | Executable analytical queries Q1-Q6 (2/4)

```
058 |           AS avg_price_per_sqm,
059 |       count(*) AS transaction_volume
060 | FROM bucketed AS b
061 | JOIN hdb_flat AS f USING (flat_id)
062 | JOIN town AS t USING (town_id)
063 | GROUP BY t.town_name, b.lease_bucket
064 | ORDER BY t.town_name,
065 |          CASE b.lease_bucket
066 |              WHEN '&lt; 50 years' THEN 1
067 |              WHEN '50-69 years' THEN 2
068 |              ELSE 3
069 |          END;
070 |
071 | -- Q3. Ten towns with the largest mean 3-room floor area, followed by
072 | -- the maximum 3-room resale price for every represented year.
073 | WITH top_towns AS (
074 |     SELECT f.town_id,
075 |            avg(rt.floor_area_sqm) AS avg_floor_area_sqm
076 |     FROM resale_transaction AS rt
077 |     JOIN hdb_flat AS f USING (flat_id)
078 |     WHERE rt.flat_type = '3 ROOM'
079 |     GROUP BY f.town_id
080 |     ORDER BY avg_floor_area_sqm DESC, f.town_id
081 |     LIMIT 10
082 | )
083 | SELECT t.town_name,
084 |        round(tt.avg_floor_area_sqm, 2) AS avg_floor_area_sqm,
085 |        extract(year FROM rt.sale_month)::int AS sale_year,
086 |        max(rt.resale_price) AS highest_resale_price
087 | FROM top_towns AS tt
088 | JOIN town AS t USING (town_id)
089 | JOIN hdb_flat AS f USING (town_id)
090 | JOIN resale_transaction AS rt USING (flat_id)
091 | WHERE rt.flat_type = '3 ROOM'
092 | GROUP BY t.town_name, tt.avg_floor_area_sqm, sale_year
093 | ORDER BY tt.avg_floor_area_sqm DESC, t.town_name, sale_year;
094 |
095 | -- Q4. Compare the closest and furthest proximity classes.
096 | -- Each flat/block is classified only by its single nearest stored hawker,
097 | -- preventing a transaction from being counted once per nearby hawker.
098 | WITH nearest_hawker AS (
099 |     SELECT DISTINCT ON (flat_id)
100 |            flat_id, proximity_class, distance_m, hawker_id
101 |     FROM v_nearby_classified
102 |     ORDER BY flat_id, distance_m, hawker_id
103 | ), prices AS (
104 |     SELECT t.town_name,
105 |            rt.flat_type,
106 |            avg(rt.resale_price) FILTER (WHERE nh.proximity_class = 'near') AS avg_near,
107 |            avg(rt.resale_price) FILTER (WHERE nh.proximity_class = 'far')  AS avg_far
108 |     FROM resale_transaction AS rt
109 |     JOIN hdb_flat AS f USING (flat_id)
110 |     JOIN town AS t USING (town_id)
111 |     JOIN nearest_hawker AS nh USING (flat_id)
112 |     GROUP BY t.town_name, rt.flat_type
113 | )
114 | SELECT town_name,
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix D | Executable analytical queries Q1-Q6 (3/4)

```
115 |        flat_type,
116 |        round(avg_near, 2) AS avg_price_near_hawker,
117 |        round(avg_far, 2) AS avg_price_far_hawker,
118 |        round((avg_near - avg_far) * 100 / NULLIF(avg_far, 0), 2)
119 |            AS price_diff_percentage
120 | FROM prices
121 | WHERE avg_near IS NOT NULL
122 |   AND avg_far IS NOT NULL
123 | ORDER BY price_diff_percentage DESC, town_name, flat_type;
124 |
125 | -- Q5. Town/band prices near hawkers with fresh-market stalls.
126 | -- SGD 350,000 is used because SGD 200,000 is universal in the complete
127 | -- 1990-2026 dataset.  De-duplication is per transaction and band.
128 | WITH eligible AS (
129 |     SELECT DISTINCT rt.transaction_id,
130 |            t.town_name,
131 |            p.proximity_class,
132 |            rt.resale_price
133 |     FROM resale_transaction AS rt
134 |     JOIN hdb_flat AS f USING (flat_id)
135 |     JOIN town AS t USING (town_id)
136 |     JOIN v_nearby_classified AS p USING (flat_id)
137 |     JOIN hawker_centre AS h USING (hawker_id)
138 |     WHERE h.has_fresh_market IS TRUE
139 | ), grouped AS (
140 |     SELECT town_name,
141 |            proximity_class,
142 |            avg(resale_price) AS avg_resale_price,
143 |            count(*) AS transaction_count
144 |     FROM eligible
145 |     GROUP BY town_name, proximity_class
146 | )
147 | SELECT town_name,
148 |        proximity_class,
149 |        round(avg_resale_price, 2) AS avg_resale_price,
150 |        transaction_count
151 | FROM grouped
152 | WHERE avg_resale_price &gt; 350000
153 | ORDER BY avg_resale_price DESC, town_name, proximity_class;
154 |
155 | -- Q6 (additional). Join all six tables for a coverage audit.
156 | -- DISTINCT removes repeated transaction-band-status rows caused by multiple
157 | -- hawkers in the same group, so averages are transaction-weighted.
158 | WITH all_table_rows AS (
159 |     SELECT DISTINCT t.town_name,
160 |            pl.level_name AS proximity_class,
161 |            CASE
162 |                WHEN h.has_fresh_market IS TRUE THEN 'yes'
163 |                WHEN h.has_fresh_market IS FALSE THEN 'no'
164 |                ELSE 'unknown'
165 |            END AS market_status,
166 |            f.flat_id,
167 |            rt.transaction_id,
168 |            rt.resale_price
169 |     FROM town AS t
170 |     JOIN hdb_flat AS f USING (town_id)
171 |     JOIN resale_transaction AS rt USING (flat_id)
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix D | Executable analytical queries Q1-Q6 (4/4)

```
172 |     JOIN nearby AS n USING (flat_id)
173 |     JOIN hawker_centre AS h USING (hawker_id)
174 |     JOIN proximity_level AS pl
175 |       ON n.distance_m &gt;= pl.min_distance_m
176 |      AND (n.distance_m &lt; pl.max_distance_m
177 |           OR (pl.upper_inclusive AND n.distance_m &lt;= pl.max_distance_m))
178 | )
179 | SELECT town_name,
180 |        proximity_class,
181 |        market_status,
182 |        count(DISTINCT flat_id) AS distinct_flats,
183 |        count(DISTINCT transaction_id) AS distinct_transactions,
184 |        round(avg(resale_price), 2) AS avg_resale_price
185 | FROM all_table_rows
186 | GROUP BY town_name, proximity_class, market_status
187 | ORDER BY town_name, proximity_class, market_status;
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix E | Performance benchmark harness (1/3)

Complete timing harness for the five cumulative data sizes, six queries, excluded warm-ups, five measured repetitions, and index creation.

```
001 | \set ON_ERROR_STOP on
002 | \timing on
003 | SET search_path = housing, public;
004 |
005 | DROP TABLE IF EXISTS benchmark_transaction;
006 | DROP TABLE IF EXISTS benchmark_results;
007 | CREATE UNLOGGED TABLE benchmark_transaction
008 |     (LIKE resale_transaction INCLUDING DEFAULTS INCLUDING CONSTRAINTS);
009 | CREATE TABLE benchmark_results (
010 |     size_level       smallint NOT NULL,
011 |     transaction_rows integer NOT NULL,
012 |     query_no         smallint NOT NULL,
013 |     repetition       smallint NOT NULL,
014 |     indexed          boolean NOT NULL,
015 |     execution_ms     numeric(14,3) NOT NULL
016 | );
017 |
018 | CREATE OR REPLACE PROCEDURE run_task3_benchmark(p_size_level smallint, p_rows integer,
    |     p_indexed boolean)
019 | LANGUAGE plpgsql
020 | AS $$
021 | DECLARE
022 |     q text;
023 |     q_no smallint;
024 |     rep smallint;
025 |     started_at timestamptz;
026 |     queries text[] := ARRAY[
027 |         $q$WITH bounds AS (SELECT date_trunc('month', max(sale_month))::date latest_month
    |     FROM benchmark_transaction)
028 |            SELECT t.town_name, rt.flat_type,
    |     avg(rt.resale_price/NULLIF(rt.floor_area_sqm,0))
029 |            FROM benchmark_transaction rt JOIN hdb_flat f USING(flat_id) JOIN town t
    |     USING(town_id) CROSS JOIN bounds b
030 |            WHERE rt.flat_type IN ('4 ROOM','5 ROOM') AND rt.sale_month &gt;= b.latest_month-
    |     interval '35 months'
031 |              AND rt.sale_month &lt; b.latest_month+interval '1 month' GROUP BY t.town_name,
    |     rt.flat_type$q$,
032 |         $q$SELECT t.town_name,
033 |              CASE WHEN rt.remaining_lease_months&lt;600 THEN '&lt; 50 years' WHEN
    |     rt.remaining_lease_months&lt;840 THEN '50-69 years' ELSE '70+ years' END,
034 |              percentile_cont(0.5) WITHIN GROUP (ORDER BY rt.resale_price),
    |     avg(rt.resale_price/NULLIF(rt.floor_area_sqm,0)), count(*)
035 |            FROM benchmark_transaction rt JOIN hdb_flat f USING(flat_id) JOIN town t
    |     USING(town_id)
036 |            GROUP BY t.town_name, 2$q$,
037 |         $q$WITH top_towns AS (
038 |              SELECT f.town_id, avg(rt.floor_area_sqm) a FROM benchmark_transaction rt JOIN
    |     hdb_flat f USING(flat_id)
039 |              WHERE rt.flat_type='3 ROOM' GROUP BY f.town_id ORDER BY a DESC LIMIT 10)
040 |            SELECT t.town_name, extract(year FROM rt.sale_month), max(rt.resale_price)
041 |            FROM top_towns x JOIN town t USING(town_id) JOIN hdb_flat f USING(town_id)
042 |              JOIN benchmark_transaction rt USING(flat_id) WHERE rt.flat_type='3 ROOM'
043 |            GROUP BY t.town_name, extract(year FROM rt.sale_month)$q$,
044 |         $q$WITH nearest AS (SELECT DISTINCT ON(flat_id) flat_id, proximity_class FROM
    |     v_nearby_classified ORDER BY flat_id,distance_m,hawker_id)
045 |            SELECT t.town_name,rt.flat_type,
046 |              avg(rt.resale_price) FILTER(WHERE n.proximity_class='near'),
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix E | Performance benchmark harness (2/3)

```
047 |             avg(rt.resale_price) FILTER(WHERE n.proximity_class='far')
048 |           FROM benchmark_transaction rt JOIN hdb_flat f USING(flat_id) JOIN town t
    |     USING(town_id) JOIN nearest n USING(flat_id)
049 |           GROUP BY t.town_name,rt.flat_type$q$,
050 |         $q$SELECT t.town_name,x.proximity_class,avg(x.resale_price)
051 |           FROM (SELECT DISTINCT
    |     rt.transaction_id,rt.flat_id,rt.resale_price,p.proximity_class
052 |                 FROM benchmark_transaction rt JOIN v_nearby_classified p USING(flat_id)
053 |                 JOIN hawker_centre h USING(hawker_id) WHERE h.has_fresh_market IS TRUE) x
054 |           JOIN hdb_flat f USING(flat_id) JOIN town t USING(town_id) JOIN proximity_level p
    |      ON p.level_name=x.proximity_class
055 |           GROUP BY t.town_name,x.proximity_class HAVING avg(x.resale_price)&gt;350000$q$,
056 |         $q$SELECT t.town_name,pl.level_name,
057 |             CASE WHEN h.has_fresh_market IS TRUE THEN 'yes' WHEN h.has_fresh_market IS
    |     FALSE THEN 'no' ELSE 'unknown' END,
058 |             count(DISTINCT f.flat_id),count(DISTINCT
    |     rt.transaction_id),avg(rt.resale_price)
059 |           FROM town t JOIN hdb_flat f USING(town_id) JOIN benchmark_transaction rt
    |     USING(flat_id)
060 |             JOIN nearby n USING(flat_id) JOIN hawker_centre h USING(hawker_id)
061 |             JOIN proximity_level pl ON n.distance_m&gt;=pl.min_distance_m AND
    |     (n.distance_m&lt;pl.max_distance_m OR (pl.upper_inclusive AND
    |     n.distance_m&lt;=pl.max_distance_m))
062 |           GROUP BY t.town_name,pl.level_name,3$q$
063 |     ];
064 | BEGIN
065 |     FOR q_no IN 1..array_length(queries, 1) LOOP
066 |         q := queries[q_no];
067 |         EXECUTE 'SELECT count(*) FROM (' || q || ') warmup';
068 |         FOR rep IN 1..5 LOOP
069 |             started_at := clock_timestamp();
070 |             EXECUTE 'SELECT count(*) FROM (' || q || ') measured';
071 |             INSERT INTO benchmark_results VALUES
072 |                 (p_size_level,p_rows,q_no,rep,p_indexed,
073 |                  extract(epoch FROM clock_timestamp()-started_at)*1000);
074 |         END LOOP;
075 |     END LOOP;
076 | END $$;
077 |
078 | DO $$
079 | DECLARE
080 |     n integer;
081 |     level_no smallint := 0;
082 | BEGIN
083 |     FOREACH n IN ARRAY ARRAY[278014,643914,696113,733266,976441] LOOP
084 |         level_no := level_no + 1;
085 |         TRUNCATE benchmark_transaction;
086 |         INSERT INTO benchmark_transaction
087 |         SELECT * FROM resale_transaction ORDER BY transaction_id LIMIT n;
088 |         ANALYZE benchmark_transaction;
089 |         CALL run_task3_benchmark(level_no,n,false);
090 |     END LOOP;
091 | END $$;
092 |
093 | CREATE INDEX benchmark_month_type_idx ON benchmark_transaction(sale_month,flat_type);
094 | CREATE INDEX benchmark_flat_idx ON benchmark_transaction(flat_id);
095 | CREATE INDEX benchmark_lease_idx ON benchmark_transaction(remaining_lease_months);
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix E | Performance benchmark harness (3/3)

```
096 | ANALYZE benchmark_transaction;
097 | CALL run_task3_benchmark(5::smallint,976441,true);
098 |
099 | \copy (SELECT size_level,transaction_rows,query_no,indexed,round(avg(execution_ms),3) AS
    |     mean_ms,round(stddev_samp(execution_ms),3) AS sd_ms,min(execution_ms) AS
    |     min_ms,max(execution_ms) AS max_ms FROM benchmark_results GROUP BY
    |     size_level,transaction_rows,query_no,indexed ORDER BY indexed,size_level,query_no) TO
    |     'benchmark_summary.csv' WITH (FORMAT csv, HEADER true)
100 |
101 | SELECT indexed, count(*) AS observations
102 | FROM benchmark_results
103 | GROUP BY indexed
104 | ORDER BY indexed;
105 |
106 | SELECT size_level, transaction_rows, query_no, indexed,
107 |        round(avg(execution_ms),3) AS mean_ms,
108 |        round(stddev_samp(execution_ms),3) AS sd_ms
109 | FROM benchmark_results
110 | GROUP BY size_level, transaction_rows, query_no, indexed
111 | ORDER BY indexed, size_level, query_no;
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Appendix F | Indexed benchmark continuation

Complete continuation script used after the type-cast correction to collect the 30 indexed measurements and export the successful benchmark summary.

```
001 | \set ON_ERROR_STOP on
002 | \timing on
003 | SET search_path = housing, public;
004 |
005 | -- The unindexed 150 observations are already present from the completed DO block.
006 | -- Remove any partial indexed observations so this resume step is repeatable.
007 | DELETE FROM benchmark_results WHERE indexed IS TRUE;
008 | CALL run_task3_benchmark(5::smallint, 976441, true);
009 |
010 | \copy (SELECT size_level,transaction_rows,query_no,indexed,round(avg(execution_ms),3) AS
    |     mean_ms,round(stddev_samp(execution_ms),3) AS sd_ms,min(execution_ms) AS
    |     min_ms,max(execution_ms) AS max_ms FROM benchmark_results GROUP BY
    |     size_level,transaction_rows,query_no,indexed ORDER BY indexed,size_level,query_no) TO
    |     'benchmark_summary.csv' WITH (FORMAT csv, HEADER true)
011 |
012 | SELECT indexed, count(*) AS observations
013 | FROM benchmark_results
014 | GROUP BY indexed
015 | ORDER BY indexed;
016 |
017 | SELECT size_level, transaction_rows, query_no, indexed,
018 |        round(avg(execution_ms),3) AS mean_ms,
019 |        round(stddev_samp(execution_ms),3) AS sd_ms
020 | FROM benchmark_results
021 | GROUP BY size_level, transaction_rows, query_no, indexed
022 | ORDER BY indexed, size_level, query_no;
```

Appendix display lines are numbered and may wrap for printing. The executable .sql file in the submission package is unchanged.

Conclusions and submission notes

The five required questions are answered using a single, reproducible database scope. Q1 alone applies the assignment's 36-month restriction; Q2-Q5 use all matched transactions. Q6 is clearly labelled as additional analysis. The housing schema, query outputs and 180 performance observations were all verified on PostgreSQL 18 using the current 976,441-transaction snapshot.

Query	Requirement satisfied	Verified output
Q1	36-month window; 4-room and 5-room; mean SGD/m²	52 groups
Q2	Three lease buckets; median, mean SGD/m², volume by town	64 groups
Q3	Top ten mean 3-room areas; yearly maximum price	369 rows
Q4	Nearest hawker only; near-versus-far percentage difference	75 comparable groups
Q5	Fresh-market hawkers; three bands; SGD 350,000 threshold	17 groups / 9 towns
Q6	All six tables; explicit yes/no/unknown market status	154 groups

Interpretation limits

Prices are nominal and not inflation-adjusted. Straight-line distance omits the street network and travel barriers. The current HDB coordinate file excludes unmatched historic addresses, creating survivorship bias. All results are descriptive aggregates and should not be interpreted causally.

Files supplied with this report

1. **SD6103_PostgreSQL_Import_Package** - schema, packaged data, verification, Q1-Q6, and benchmark scripts in replay order.
2. **SD6103_Task3_Queries.sql** - standalone executable PostgreSQL for Q1-Q6, using the supplied ER-model names.
3. **SD6103_Task3_Result_CSVs.zip** - full result tables, including all 369 Q3 rows and all 154 Q6 groups.
4. **SD6103_Task3_Benchmark_Summary.csv** - current PostgreSQL timing summary for 150 unindexed and 30 indexed observations.
5. **SD6103_Task3_Completed_Final_Verified.pdf** - narrative, pgAdmin evidence, current performance figures, and the complete reproducibility code appendix.